Localizzazione in ambiente dinamico

Simulazione interattiva con EKF e gate adattativo

Indice

1 Obiettivo e impostazione del progetto **1**

2 Ambiente e sensore simulati **2**
2.1 Mappa statica e persone mobili . . . 2
2.2 Misure Lidar . . . 2

3 Modello del robot e filtro EKF **3**
3.1 Stato e modello cinematico . . . 3
3.2 Predizione . . . 3
3.3 Modello di osservazione e correzione . . . 3

4 Associazione delle misure e gate dinamico **4**
4.1 Controllo geometrico . . . 4
4.2 Controllo sull'innovazione . . . 4

5 Sessione analizzata e risultati **5**
5.1 Provenienza dei dati e riproducibilità . . . 5
5.2 Localizzazione . . . 5
5.3 Classificazione delle misure . . . 7
5.4 Interfaccia interattiva . . . 7

6 Discussione e conclusioni **8**

7 Estratti dell'implementazione **9**
7.1 Configurazione . . . 9
7.2 Costruzione della mappa e inizializzazione . . . 10
7.3 Modello cinematico e movimento delle persone . . . 10
7.4 Misure e trasformazioni . . . 10
7.5 Predizione e aggiornamento EKF . . . 11
7.6 Associazione e soglia adattativa . . . 11

1 Obiettivo e impostazione del progetto

Il progetto realizza una simulazione interattiva di localizzazione di un robot in una mappa nota contenente landmark statici e persone mobili. Il robot viene guidato mediante tastiera; un Extended Kalman Filter (EKF) ne stima la posizione e l'orientamento utilizzando odometria rumorosa e misure Lidar simulate di distanza e angolo.

Per evitare che osservazioni di persone mobili producano correzioni errate, ogni misura viene confrontata con la mappa statica. Un controllo geometrico e un gate adattativo sull'innovazione determinano se utilizzarla nell'EKF oppure scartarla. Una misura rifiutata è incompatibile con la mappa: questo non costituisce, da solo, una prova del movimento dell'oggetto.

La valutazione riguarda il comportamento del sistema con gate dinamico durante una sessione manuale, senza confronti con altre strategie.

2 Ambiente e sensore simulati

2.1 Mappa statica e persone mobili

La mappa contiene punti di riferimento noti disposti lungo due file orizzontali e in posizioni isolate. La disposizione effettivamente implementata è riportata nell'appendice, estratta automaticamente dalla funzione `_make_static_objects` dello script.

Le posizioni delle persone sono inizializzate con distribuzione uniforme nell'intervallo $[-0.65W, 0.65W]$ lungo ciascun asse, dove W è la semi-estensione del mondo. Le due componenti della velocità iniziale sono estratte indipendentemente da una distribuzione normale. Il secondo argomento di `rng.normal` rappresenta la deviazione standard, non la varianza; i valori utilizzati sono consultabili nell'estratto di inizializzazione in appendice.

A ogni passo si applica:

$$p_{k+1} = p_k + v_k \Delta t.$$

Al superamento dei limiti $[-0.8W, 0.8W]$ viene invertita la componente della velocità relativa all'asse interessato; la posizione viene ricondotta entro il limite tramite clipping.

2.2 Misure Lidar

Il sensore è modellato a livello di oggetti puntiformi: produce misure rumorose per i punti entro la portata, senza simulare una scansione laser completa, la segmentazione o le occlusioni. Per un landmark $l = (l_x, l_y)^T$ e una posa $x = (x, y, \theta)^T$:

$$\begin{aligned} \Delta_x &= l_x - x, & \Delta_y &= l_y - y, \\ r &= \sqrt{\Delta_x^2 + \Delta_y^2}, & \beta &= \mathrm{wrap}(\mathrm{atan2}(\Delta_y, \Delta_x) - \theta). \end{aligned}$$

Il vettore di misura è $z = (r, \beta)^T$. Alla distanza e all'angolo si aggiungono rumori gaussiani indipendenti, con covarianza:

$$R = \mathrm{diag}(\sigma_r^2, \sigma_\beta^2).$$

La funzione `wrap_angle` riporta gli angoli nell'intervallo $[-\pi, \pi)$.

3 Modello del robot e filtro EKF

3.1 Stato e modello cinematico

Lo stato stimato è $x_k = (x_k, y_k, \theta_k)^T$. Il modello uniciclo discretizzato è:

$$\begin{aligned} x_{k+1} &= x_k + v_k \cos(\theta_k)\Delta t, \\ y_{k+1} &= y_k + v_k \sin(\theta_k)\Delta t, \\ \theta_{k+1} &= \mathrm{wrap}(\theta_k + \omega_k \Delta t). \end{aligned}$$

Il moto reale utilizza i comandi impartiti dalla tastiera; all'EKF vengono forniti comandi perturbati da rumore per rappresentare errori di odometria. La posizione reale viene inoltre limitata ai confini del mondo simulato tramite clipping.

Il filtro è inizializzato con uno scostamento intenzionale dalla posa reale e con una covarianza diagonale. Le condizioni iniziali della sessione analizzata sono ricavate dal primo campione del log, non assunte uguali a quelle di altre prove.

3.2 Predizione

La predizione della posa segue il modello cinematico. La covarianza viene propagata mediante:

$$P_{k+1}^{-} = F_k P_k^{+} F_k^T + Q,$$

con:

$$F_k = \begin{pmatrix} 1 & 0 & -v_k \sin(\theta_k)\Delta t \\ 0 & 1 & v_k \cos(\theta_k)\Delta t \\ 0 & 0 & 1 \end{pmatrix}, \qquad Q = \mathrm{diag}(\sigma_{xy}^2, \sigma_{xy}^2, \sigma_\theta^2).$$

Q rappresenta la covarianza del rumore di processo per passo, adottata nell'implementazione; non coincide con il rumore casuale aggiunto ai comandi.

3.3 Modello di osservazione e correzione

Il modello $h(x, l)$ restituisce distanza e angolo previsti. Per $q = \Delta_x^2 + \Delta_y^2$ e $r = \sqrt{q}$, il suo Jacobiano rispetto alla posa è:

$$H = \begin{pmatrix} -\Delta_x/r & -\Delta_y/r & 0 \\ \Delta_y/q & -\Delta_x/q & -1 \end{pmatrix}.$$

Nel codice si impone un piccolo limite inferiore a q per evitare divisioni per zero. Per una misura accettata, indicando con $-$ e $+$ le quantità prima e dopo la correzione:

$$\begin{aligned} \nu_k &= z_k - h(\hat{x}_k^{-}), \\ S_k &= H_k P_k^{-} H_k^T + R, \\ K_k &= P_k^{-} H_k^T S_k^{-1}, \\ \hat{x}_k^{+} &= \hat{x}_k^{-} + K_k \nu_k. \end{aligned}$$

La componente angolare dell'innovazione e l'orientamento corretto vengono normalizzati. La covarianza viene aggiornata con la forma di Joseph:

$$P_k^+ = (I - K_k H_k) P_k^- (I - K_k H_k)^T + K_k R K_k^T.$$

Questa formulazione favorisce la stabilità numerica. S_k è la covarianza dell'innovazione e K_k il guadagno di Kalman. Le misure accettate vengono elaborate sequenzialmente.

4 Associazione delle misure e gate dinamico

4.1 Controllo geometrico

La misura viene convertita nel riferimento locale del robot:

$$p_{\text{obs}} = \begin{pmatrix} r\cos\beta \\ r\sin\beta \end{pmatrix}.$$

Per ciascun landmark si predice la posizione relativa:

$$p_{\text{exp},i} = R(\hat{\theta}^-)^T (l_i - \hat{p}^-), \qquad R(\theta)^T = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}.$$

Il candidato è quello che minimizza $e_{\text{rel},i} = \|p_{\text{obs}} - p_{\text{exp},i}\|$. Il controllo geometrico richiede che tale errore sia inferiore a `association_gate`.

4.2 Controllo sull'innovazione

Per il candidato selezionato si verifica anche:

$$d^2 = \nu^T S^{-1} \nu < g_{\text{dyn}}.$$

La soglia adattativa è:

$$g_{\text{dyn}} = \text{clip}\left(g_0 + c_u \sqrt{\max(\det S, 0)} + c_v |\tilde{v}| - c_r r, g_{\min}, g_{\max}\right),$$

dove $\tilde{v}$ è il comando lineare rumoroso fornito al filtro. I parametri effettivi sono quelli della funzione e della configurazione riportate in appendice. A parità degli altri termini, l'incertezza e la velocità ampliano la soglia, mentre la distanza la restringe, entro i limiti del clipping. La soglia è euristica: l'adattamento non corrisponde a un livello di confidenza statistico costante.

La misura viene accettata soltanto se supera entrambi i controlli. Il termine "dinamico" riferito al gate indica che la soglia si adatta alle condizioni operative; il metodo non stima esplicitamente la velocità di ciascun oggetto.

L'associazione geometrica utilizza la posa predetta all'inizio del fotogramma. Le distanze di innovazione e le relative covarianze sono invece calcolate con lo stato corrente dell'EKF, che può includere aggiornamenti dovuti a misure già accettate nello stesso fotogramma.

L'etichetta `source`, nota al simulatore, serve esclusivamente a valutare i rifiuti e aggiornare i contatori. Non viene utilizzata per decidere l'accettazione della misura.

5 Sessione analizzata e risultati

5.1 Provenienza dei dati e riproducibilità

Il report seleziona automaticamente il CSV con timestamp più recente nel nome tra quelli presenti in `artifacts`. Per scegliere una sessione specifica si può impostare `SESSION_CSV` nel primo blocco Python del QMD. Tutti gli indicatori seguenti vengono ricalcolati dal CSV; l'eventuale JSON viene letto soltanto se ha lo stesso identificativo di sessione.

Log analizzato: lidar_dynamic_gate_log_20261016_134750_809690.csv.

Parametro della registrazione	Valore
Tempo simulato [s]	26,08
Passo temporale [s]	0,08
Campioni, inclusa la posa iniziale	327
Seed registrato nel riepilogo	7

La guida è manuale: il seed non rende riproducibile da solo la sequenza di comandi. Il log non registra gli eventi di reset del filtro; se è stato usato il tasto `R`, le metriche includono i campioni prima e dopo il reset. Per una prova senza interventi sulla stima occorre evitare tale comando.

5.2 Localizzazione

L'errore di posizione e il relativo RMSE sono definiti come:

$$e_k = \sqrt{(\hat{x}_k - x_k)^2 + (\hat{y}_k - y_k)^2}, \qquad \mathrm{RMSE}_{\mathrm{pos}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_k^2}.$$

L'errore angolare è normalizzato in $[-\pi, \pi)$ e convertito in gradi per la presentazione.

Tabella 1: Errori calcolati dalla sessione selezionata.

Indicatore	Valore
Errore iniziale di posizione [m]	0,6103
RMSE di posizione, tutti i campioni [m]	0,0432
Errore medio di posizione [m]	0,0252
Errore massimo di posizione [m]	0,6103
Errore finale di posizione [m]	0,0271
RMSE di orientamento [gradi]	0,566
RMSE di posizione senza il campione iniziale [m]	0,0269
Errore massimo senza il campione iniziale [m]	0,0741

Gli indicatori principali includono lo scostamento iniziale imposto al filtro. Gli ultimi due indicatori escludono soltanto il primo campione: non rappresentano una selezione automatica del regime stazionario.

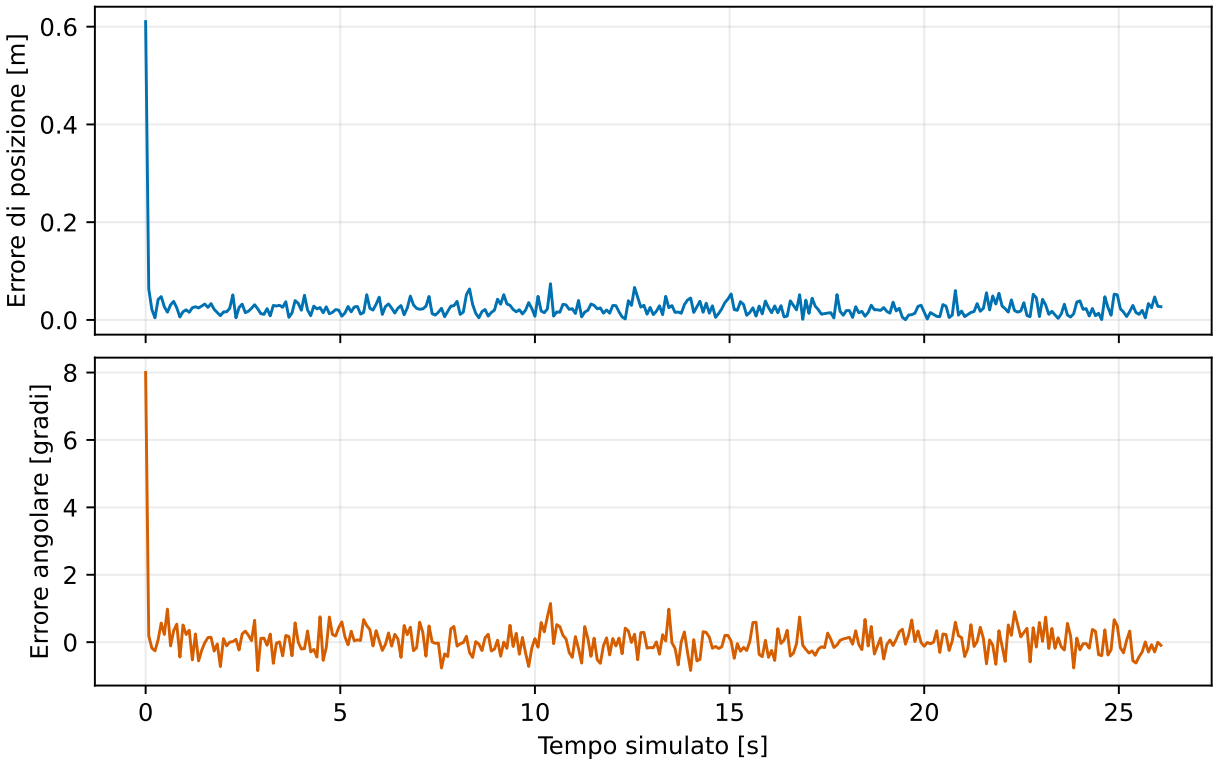


Figura 1: Errori di posizione e orientamento durante la sessione selezionata.

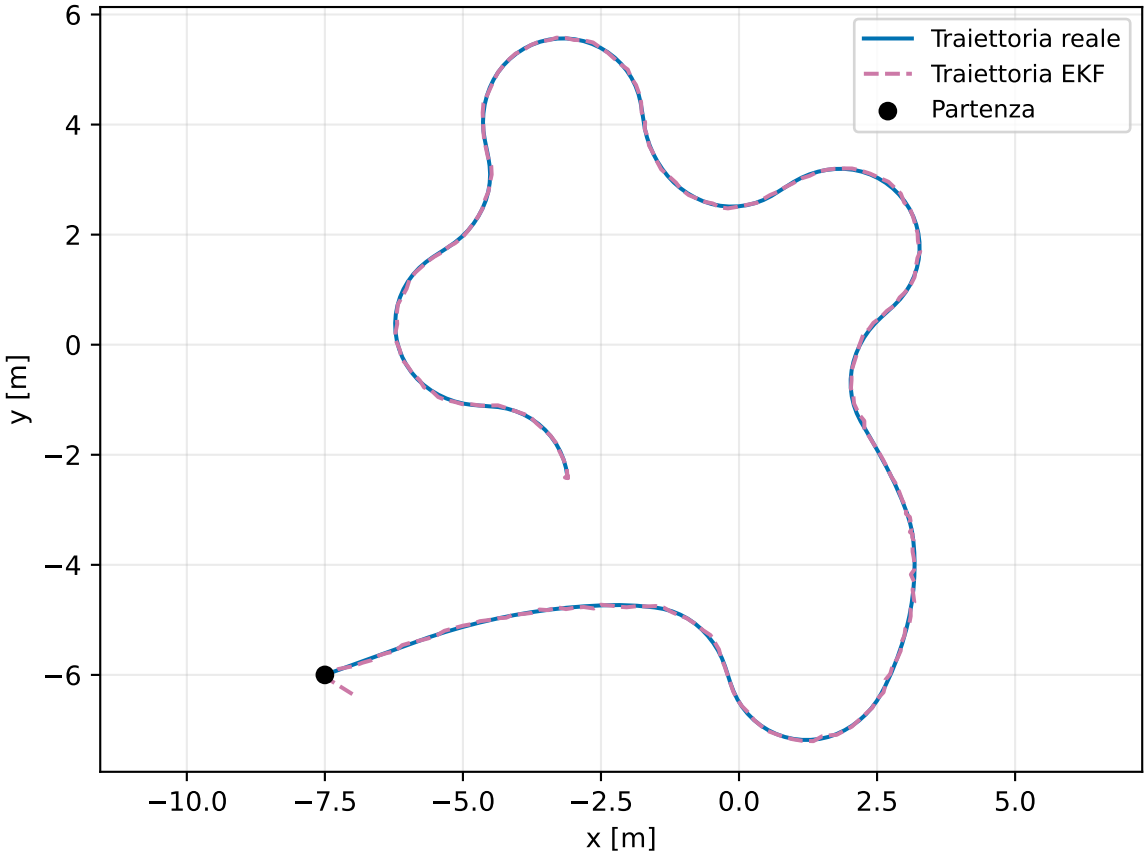


Figura 2: Confronto tra traiettoria reale e stimata, ricostruite dal CSV.

5.3 Classificazione delle misure

La provenienza reale delle misure è nota nel simulatore. Le misure generate sono sommate sui fotogrammi; i contatori cumulativi di accettazione e rifiuto vengono invece letti esclusivamente nell'ultima riga, per evitare conteggi ripetuti.

Tabella 2: Decisioni del sistema distinte per provenienza reale della misura.

Provenienza	Accettata	Scartata	Totale
Landmark statici	2331	50	2381
Persone mobili	0	550	550
Totale	**2331**	**600**	**2931**

I conteggi riguardano misure, non oggetti distinti: la stessa persona o lo stesso landmark può essere osservato ripetutamente. "Accettata" significa ritenuta compatibile con la mappa, non necessariamente proveniente da un oggetto realmente statico.

Indicatore	Definizione	Valore
Rifiuto delle misure dinamiche (recall)	Dinamiche scartate / dinamiche generate	100,00%
Precisione degli scarti	Dinamiche scartate / tutte le scartate	91,67%
Rifiuto delle misure statiche	Statiche scartate / statiche generate	2,10%

Delle 550 misure provenienti da persone mobili, 550 sono state scartate e 0 sono state accettate nell'EKF. La percentuale di rifiuto delle misure dinamiche è 100,00%.

Delle 2381 misure statiche, 50 sono state scartate. Questo conteggio quantifica il costo del filtraggio in termini di osservazioni statiche non utilizzate.

5.4 Interfaccia interattiva

I quadrati grigi rappresentano i landmark noti; i contorni verdi indicano i landmark associati alle misure accettate. Le persone sono arancioni, le misure rifiutate sono croci rosse, il robot reale è blu e la stima EKF è rosa. I comandi sono frecce o `WASD` per la guida, spazio per fermarsi, `R` per reimpostare la stima ed `Esc` per salvare uno screenshot e chiudere la finestra.

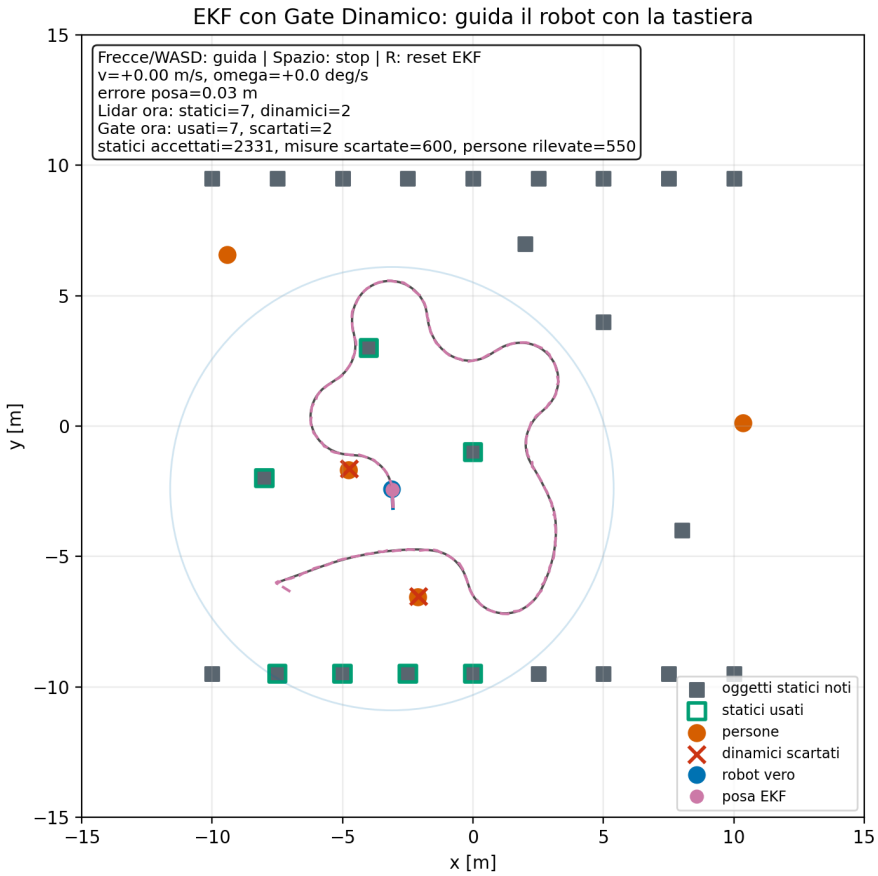


Figura 3: Esempio dell'interfaccia interattiva. Lo screenshot è illustrativo e non viene utilizzato per calcolare le metriche.

Lo screenshot ha un nome fisso e non contiene l'identificativo della sessione nel nome: non se ne assume quindi la corrispondenza con il CSV selezionato. I risultati quantitativi e le traiettorie precedenti dipendono esclusivamente dal log analizzato.

6 Discussione e conclusioni

La sessione analizzata presenta un RMSE di posizione pari a 0,0432 m e un errore finale pari a 0,0271 m. Sono state utilizzate 2331 misure e scartate 600 misure.

In questa prova tutte le misure delle persone sono state escluse dall'aggiornamento dell'EKF.

Il rifiuto delle misure statiche è pari a 2,10%. Il risultato deve essere interpretato insieme alla capacità di rifiutare le osservazioni dinamiche, poiché un filtro troppo restrittivo può eliminare anche informazioni utili alla localizzazione.

La valutazione documenta il comportamento di una singola sessione simulata; non dimostra superiorità rispetto ad altri metodi né garantisce le stesse prestazioni in qualsiasi ambiente. I principali limiti sono la semplicità del modello del sensore e delle persone, la disponibilità di una mappa nota e la scelta euristica delle soglie.

La posa vera permette di misurare direttamente l'errore: su un robot reale sarebbe necessario un riferimento indipendente. Il progetto mostra l'integrazione di associazione geometrica, gate adattativo ed EKF in un'interfaccia guidabile, con risultati verificabili tramite log.

7 Estratti dell'implementazione

I frammenti seguenti vengono generati dal file `interactive_lidar_dynamic_gate.py` presente al momento del render. La logica è quella dello script; commenti e docstring vengono rimossi e la formattazione viene normalizzata. Il file di simulazione non viene eseguito per produrre questi estratti.

Il log attuale non conserva una copia completa della configurazione o del codice della sessione: questi estratti documentano quindi lo script disponibile, non certificano che una vecchia registrazione sia stata prodotta dalla medesima versione. Dopo modifiche al modello o ai parametri è opportuno registrare una nuova sessione.

7.1 Configurazione

```python
@dataclass
class ExperimentConfig:
    seed: int = 7
    dt: float = 0.1
    steps: int = 550
    world_size: float = 18.0
    n_landmarks: int = 42
    n_people: int = 6
    lidar_max_range: float = 11.0
    lidar_fov: float = 2.0 * np.pi
    sigma_range: float = 0.12
    sigma_bearing: float = np.deg2rad(1.8)
    process_sigma_xy: float = 0.035
    process_sigma_theta: float = np.deg2rad(1.2)
    base_gate: float = 5.991
    min_gate: float = 3.2
    max_gate: float = 9.5
    gate_speed_gain: float = 0.45
    gate_uncertainty_gain: float = 0.7
    gate_range_tightening: float = 0.025
```

```python
@dataclass
class InteractiveConfig(ExperimentConfig):
    dt: float = 0.08
    world_size: float = 14.0
    n_landmarks: int = 24
    n_people: int = 4
    lidar_max_range: float = 8.5
    sigma_range: float = 0.06
    sigma_bearing: float = np.deg2rad(1.0)
    relative_motion_gate: float = 0.55
    association_gate: float = 1.1
```

7.2 Costruzione della mappa e inizializzazione

```
def _make_static_objects(self) -> np.ndarray:
    wall_top = np.column_stack((np.linspace(-10, 10, 9), np.full(9, 9.5)))
    wall_bottom = np.column_stack((np.linspace(-10, 10, 9), np.full(9, -9.5)))
    columns = np.array([[-8, -2], [-4, 3], [0, -1], [5, 4], [8, -4], [2, 7]],
 ↪  dtype=float)
    return np.vstack((wall_top, wall_bottom, columns))
```

```
self.robot = np.array([-7.5, -6.0, np.deg2rad(20.0)])
self.people = self.rng.uniform(-0.65 * self.cfg.world_size, 0.65 * self.cfg.world_size,
 ↪  size=(self.cfg.n_people, 2))
self.people_vel = self.rng.normal(0.0, 0.55, size=(self.cfg.n_people, 2))
self.ekf = PoseEKF(initial_state=self.robot + np.array([0.5, -0.35, np.deg2rad(8.0)]),
 ↪  initial_cov=np.diag([0.7, 0.7, np.deg2rad(10.0)]) ** 2, cfg=self.cfg)
```

7.3 Modello cinematico e movimento delle persone

```
def step_unicycle(state: np.ndarray, control: tuple[float, float], dt: float) ->
 ↪  np.ndarray:
    v, omega = control
    x, y, th = state
    return np.array([x + v * np.cos(th) * dt, y + v * np.sin(th) * dt, wrap_angle(th +
     ↪  omega * dt)])
```

```
def _move_people(self) -> None:
    self.people += self.people_vel * self.cfg.dt
    limit = 0.8 * self.cfg.world_size
    for i in range(len(self.people)):
        for axis in range(2):
            if self.people[i, axis] < -limit or self.people[i, axis] > limit:
                self.people_vel[i, axis] *= -1.0
                self.people[i, axis] = np.clip(self.people[i, axis], -limit, limit)
```

7.4 Misure e trasformazioni

```
def relative_xy(robot_state: np.ndarray, world_point: np.ndarray) -> np.ndarray:
    dx = world_point[0] - robot_state[0]
    dy = world_point[1] - robot_state[1]
    c = np.cos(robot_state[2])
    s = np.sin(robot_state[2])
    return np.array([c * dx + s * dy, -s * dx + c * dy])
```

```
def relative_to_measurement(rel_xy: np.ndarray, rng: np.random.Generator, cfg:
 ↪  InteractiveConfig) -> np.ndarray:
    r = np.hypot(rel_xy[0], rel_xy[1])
    b = np.arctan2(rel_xy[1], rel_xy[0])
    return np.array([r + rng.normal(0.0, cfg.sigma_range), wrap_angle(b +
     ↪  rng.normal(0.0, cfg.sigma_bearing))])
```

```
def measurement_to_relative(z: np.ndarray) -> np.ndarray:
    return np.array([z[0] * np.cos(z[1]), z[0] * np.sin(z[1])])
```

```
def measurement_model(self, landmark: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    dx = landmark[0] - self.x[0]
    dy = landmark[1] - self.x[1]
    q = max(dx * dx + dy * dy, 1e-09)
    r = np.sqrt(q)
    bearing = wrap_angle(np.arctan2(dy, dx) - self.x[2])
    H = np.array([[-dx / r, -dy / r, 0.0], [dy / q, -dx / q, -1.0]])
    return (np.array([r, bearing]), H)
```

7.5 Predizione e aggiornamento EKF

```
def predict(self, control: tuple[float, float]) -> None:
    v, omega = control
    x, y, th = self.x
    dt = self.cfg.dt
    self.x = np.array([x + v * np.cos(th) * dt, y + v * np.sin(th) * dt, wrap_angle(th
    ↪  + omega * dt)])
    F = np.array([[1.0, 0.0, -v * np.sin(th) * dt], [0.0, 1.0, v * np.cos(th) * dt],
↪   [0.0, 0.0, 1.0]])
    Q = np.diag([self.cfg.process_sigma_xy ** 2, self.cfg.process_sigma_xy ** 2,
↪   self.cfg.process_sigma_theta ** 2])
    self.P = F @ self.P @ F.T + Q
```

```
def innovation_distance(self, z: np.ndarray, landmark: np.ndarray, R: np.ndarray) ->
↪   tuple[float, np.ndarray]:
    z_hat, H = self.measurement_model(landmark)
    innovation = z - z_hat
    innovation[1] = wrap_angle(innovation[1])
    S = H @ self.P @ H.T + R
    d2 = float(innovation.T @ np.linalg.inv(S) @ innovation)
    return (d2, S)
```

```
def update(self, z: np.ndarray, landmark: np.ndarray, R: np.ndarray) -> float:
    z_hat, H = self.measurement_model(landmark)
    innovation = z - z_hat
    innovation[1] = wrap_angle(innovation[1])
    S = H @ self.P @ H.T + R
    K = self.P @ H.T @ np.linalg.inv(S)
    self.x = self.x + K @ innovation
    self.x[2] = wrap_angle(self.x[2])
    I = np.eye(3)
    self.P = (I - K @ H) @ self.P @ (I - K @ H).T + K @ R @ K.T
    return float(innovation.T @ np.linalg.inv(S) @ innovation)
```

7.6 Associazione e soglia adattativa

```
def _classify_and_update(self, measurements: list[dict], noisy_control: tuple[float,
↪  float]) -> None:
    R = np.diag([self.cfg.sigma_range ** 2, self.cfg.sigma_bearing ** 2])
    accepted_static = []
    dynamic_world = []
    predicted_pose = self.ekf.x.copy()
    for measurement in measurements:
        z = measurement['z']
        observed_rel = measurement_to_relative(z)
        candidates = []
        for i, landmark in enumerate(self.landmarks):
            expected_rel = relative_xy(predicted_pose, landmark)
            rel_error = np.linalg.norm(observed_rel - expected_rel)
            d2, S = self.ekf.innovation_distance(z, landmark, R)
            gate = self._dynamic_motion_gate(S, z[0], abs(noisy_control[0]))
            candidates.append((rel_error, d2, gate, i))
        rel_error, d2, gate, best_idx = min(candidates, key=lambda item: item[0])
        is_static_consistent = rel_error < self.cfg.association_gate and d2 < gate
        if is_static_consistent:
            self.ekf.update(z, self.landmarks[best_idx], R)
            accepted_static.append(self.landmarks[best_idx])
            self.accepted_count += 1
        else:
            dynamic_world.append(self._measurement_world_point(z))
            self.rejected_count += 1
            if measurement['source'] == 'person':
                self.dynamic_count += 1
    self.accepted_static_world = np.array(accepted_static) if accepted_static else
    ↪  np.empty((0, 2))
    self.detected_dynamic = np.array(dynamic_world) if dynamic_world else np.empty((0,
    ↪  2))
    self.current_gate_static = len(accepted_static)
    self.current_gate_dynamic = len(dynamic_world)
```

```
def _dynamic_motion_gate(self, S: np.ndarray, measured_range: float, speed: float) ->
↪  float:
    uncertainty = np.sqrt(max(np.linalg.det(S), 0.0))
    threshold = self.cfg.base_gate + self.cfg.gate_uncertainty_gain * uncertainty +
↪  self.cfg.gate_speed_gain * speed - self.cfg.gate_range_tightening * measured_range
    return float(np.clip(threshold, self.cfg.min_gate, self.cfg.max_gate))
```